

BT Middleware — software as infrastructure

J Calladine

BT Middleware has been in production use since 1987. Every working day a user population of 60 000 people execute 25 million transactions on systems that generate in excess of 70% of BT's revenues. BT Middleware is installed on 32 production mainframe instances.

BT Middleware provides both a development and run-time environment for applications that is controlled and manageable. Application standards can be enforced providing reuse of common code as well as common 'look and feel' user interfaces. The use of BT Middleware has allowed the rapid implementation of leading edge solutions to pressing business requirements and Middleware is evolving to allow BT's operational support systems (OSS) to participate seamlessly in complex distributed architectures.

The future for BT's OSS lies in their ability to integrate with new applications developed on open systems platforms. BT Middleware has been evolving to provide the required interconnection capabilities and now supports numerous batch and on-line interfaces via strategic infrastructure products. This paper describes the major Middleware infrastructure components developed and supported by BT's London Software Engineering Centre that allow the company's huge investment in OSS to be exploited in today's environment.

1. Introduction

Two related papers [1, 2] describe two of BT's primary operational support systems (OSS) — CSS (customer service system) and COSMOSS (a common Middleware system). These systems support the largest user populations in BT and account for a large percentage of BT's OSS computing. Both systems were built using BT Middleware products and use these products to interconnect with other OSS applications.

BT Middleware provides both a development and run-time environment for applications that is controlled and manageable. Application standards can be enforced providing reuse of common code as well as common 'look and feel' user interfaces. The use of BT Middleware has allowed the rapid implementation of leading-edge solutions to pressing business requirements and Middleware is evolving to allow BT's OSS to participate seamlessly in complex distributed architectures.

The Middleware products described in this paper are:

- common middleware (CMW) — the application development and run-time environment,
- transaction switching (part of CMW) — the seamless integration of 29 physical CSS databases into a single logical system,

- middleware message-based interface (MMBI) (part of CMW) — the ability to access existing screen-based transactions in a client/server manner,
- systems interconnect service shell (SISS) — simplified MVS/Unix communications solution,
- generalised message interface (GMI) — a standard application programming interface (API) for distributing Middleware applications,
- balanced distribution service (BDS) — a managed file transfer solution for all BT application platforms.

With the exception of the BDS, these components are all used in the development of large on-line transaction processing systems and in the access of these systems by other applications. The components fit together as shown in Fig 1.

The important thing to note is that the applications are removed both from the specifics of the operating system and also from the complexity of network access and programming. It is this separation that has allowed Middleware systems to evolve from monolithic to distributed architectures and will continue to allow new technologies to be absorbed and exploited while preserving

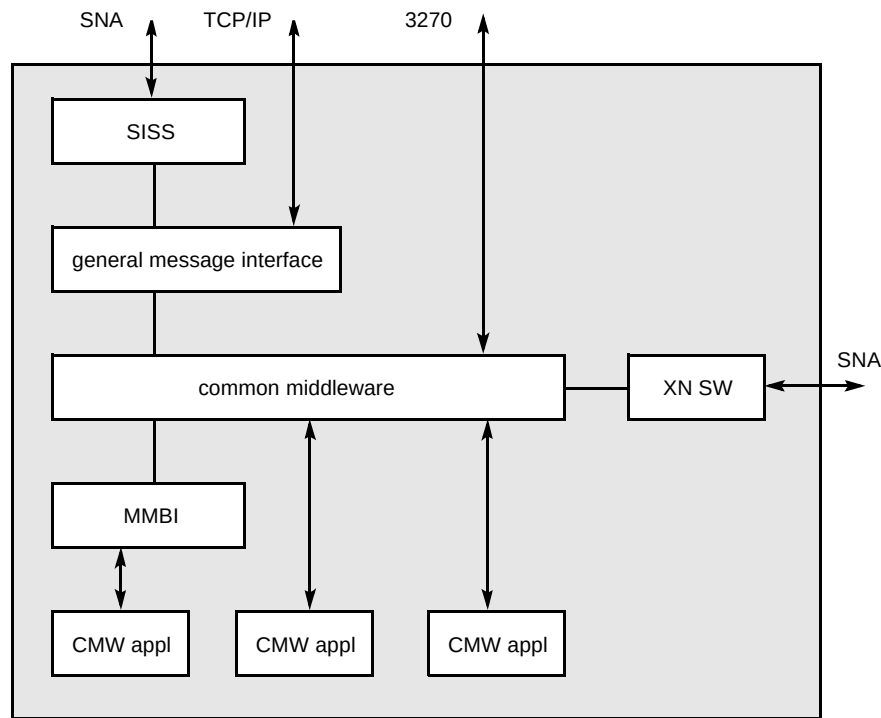


Fig 1 The relationship of BT's Middleware components.

and integrating the existing functionality already developed at great cost.

2. BT Middleware

BT has a reasonable claim to be the first user of the term 'Middleware'. The term was used in BT as long ago as 1984 when BT and Logica began to design the CSS application. From the start it was known that CSS would be one of the largest and most comprehensive computer applications ever written. It was felt that the unconstrained development of such a system by some 300 programmers would be to invite disaster unless controls could be enforced.

What was needed was an environment that would isolate the programmer from the complexities of the transaction processing monitor (CICS) and provide common routines, utilities and sub-systems to be shared between applications. BT Middleware was thus designed to simplify application development and allow optimal use of programmers to be able to produce a highly integrated system conforming to a single application structure and presentation standard.

It should be noted that in 1984 there was no option but to build this Middleware in-house. There was not a commercially available alternative that could be selected and there are few products that even come close in the market-place today. The construction of BT Middleware was a major undertaking that has been justified several

times over by the flexibility in implementing new developments.

BT Middleware was originally developed for the multiple virtual storage (MVS) CICS and integrated database management system (IDMS) DB environment. It was subsequently ported to ICL VME and also enhanced to allow DB2 database access. The MVS CICS version of Middleware is now known as common middleware (CMW). The original components of BT Middleware included:

- dialogue manager,
- static and dynamic tables,
- access control,
- menu management,
- validation,
- error handling/abend control,
- message service,
- system statistics,
- report printing,
- background schedulers.

This list of components has been expanded to include:

- routing process (see section 3),
- MMBI server (see section 5),
- GMI (see section 6).

Dialogue manager is the main Middleware component. Each time a terminal sends data the dialogue manager is executed. Dialogue manager controls the processing of the input, calling the various Middleware components and application programs as necessary. Figure 2 positions dialogue manager with the other components of CMW.

The CMW tables allow each CMW system to be configured uniquely. Dynamic-tables load allows these configuration changes to be introduced while the application is running. Validation is a large part of the processing of any terminal input. Within the CMW tables is a set of tables that specify the basic validation to be performed on every field on every screen. Before the terminal input is passed to the application program each field on the screen is validated according to the rules in the tables.

CSS was never a single application. It contains a total of 50 sub-systems including order entry, order handling, fault repair, billing, frame management, etc. These components of Middleware did not have to be written for each and every business function on CSS, thus speeding up the development as well as enforcing a development standard. The Middleware components are depicted in Fig 2.

The Middleware applications include:

- CSS [1],
- COSMOSS [2],
- logistics (supply chain) — a CICS DB2 application,

- Tardis — an ICL VME system providing a fixed-asset register for BT.

2.1 Middleware summary

The original objectives of BT Middleware have been met and the success of the product proven with CSS in its tenth year and enjoying high levels of availability and service because of the Middleware environment. The success of Middleware can also be measured by its use for other projects needing the same benefits of code reuse and development consistency.

3. Transaction switching

Most readers of this paper will be able to recall a time when an inquiry to BT's operator services to check a bill or to report a fault might get the response: "I'm sorry but I cannot handle this call; you must contact your own district." The reason for this state of affairs was that CSS was implemented on a district basis with each CSS image looking after the data for that region's customers only. The sheer volume of data that needed to be held for CSS prevented a single image system being implemented. The downside of having a regionally organised CSS was that customers could not, for example, call in from work in London to make an enquiry on their home account in Surrey.

In 1991 this 'shunting' of CSS customers on to another contact point became a thing of the past with the introduction of transaction switching.

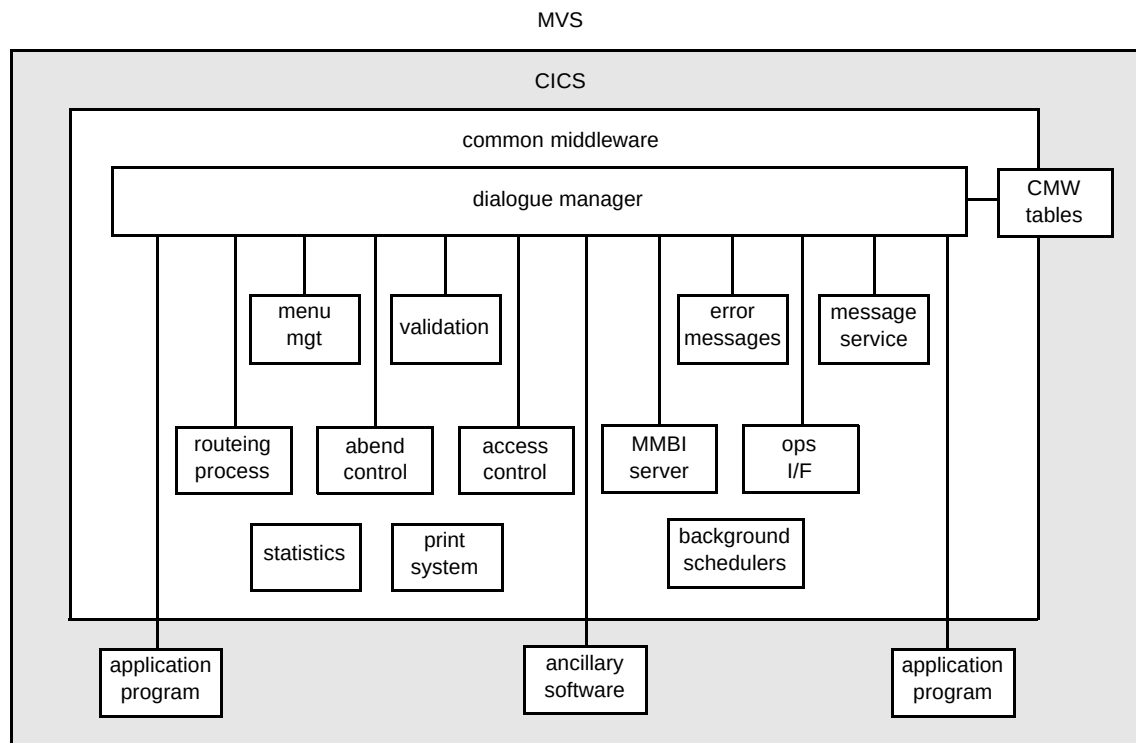


Fig 2 The positioning of dialogue manager with other CMW components.

3.1 Seamless access

Transaction switching exploits the fact that all screen input/output in a CMW system is handled by the dialogue manager component. The application itself is not responsible for receiving data from or sending data to the remote device. This feature allowed the Middleware developers to insert 'routeing' processes in between the screen input being processed by Middleware and the application program being invoked.

In this way, it is possible for Middleware itself to determine that the customer's account is not resident on this system and dynamically route the screen input to the system that owns the relevant data. The separate CSS systems are joined together using SNA LU6.2 network protocol and distributed transaction processing techniques.

Certain CSS transactions are used to select the correct database location by key data input. These are known as switch initiating transactions. In addition to these there are 'follow my leader' transactions that will automatically switch to the database used by the previous transaction.

CSS Middleware transactions can be switched on the following customer data:

- telephone number,
- account number,
- address elements:
 - postcode,
 - post town,
 - locality,
- charge-card number,
- order number.

This extra processing and network switching adds an overhead of approximately 0.2 sec to the overall response time.

3.2 Switching today

When transaction switching was first designed it was to address the requirement for a London front office. The benefits of transaction switching were soon recognised by the business as a whole and today all CSS systems are connected to every other CSS system. This has provided the seamless nature of CSS that has come to be accepted as normal.

At present the vast majority of CSS transactions are able to be switched. Every day some 8-9 million transactions

(about 40% of the total) are switched to run in a remote CSS.

3.3 Business benefits

The business benefits of transaction switching have been enormous. A major benefit is that it was introduced transparently from both the users' and application developers' points of view. Application programs did not need to be re-engineered to allow this way of working so the implementation time required to deliver this solution was far less than could normally be expected for a project of this size and scope.

Transaction switching has enabled flexibility in the way CSS users operate. With access to all CSS systems available from a single sign-on, customer calls may be routed to a service centre anywhere in the country. There is no longer any need for geographically organised service centres and rationalisation of service centres has been undertaken.

In short, transaction switching enabled better service to be delivered to BT's customers in a short time and transparently to both users and developers alike.

4. Systems interconnect service shell (SISS)

So far this paper has discussed BT Middleware and transaction switching, which allows Middleware systems like CSS to talk to each other and provide a single point of entry for the user.

Middleware systems have never been stand-alone 'islands' of processing, however. Batch feeds, in and out, are processed daily, but, in addition to this, on-line interfacing requirements are very common. Remote open systems applications have been interfacing with Middleware systems for many years. The first few of these interfaces were very application-specific and each was implemented using a different solution. The development time and cost of providing a bespoke interface including comprehensive testing for each system meant that interfacing to Middleware systems was not a trivial task. Programming to network APIs is a complex business requiring specialist skills.

It was decided to call a halt to these bespoke interfaces and develop an off-the-shelf solution that could be reused for multiple interfaces, be delivered quickly and enable the same benefits of Middleware to be realised in the development of future interfaces.

The systems interconnect service shell was developed to provide an easy-to-use interface for on-line communication between Unix and MVS/CICS environments. Developed by BT's systems interconnect team, it provides simple-to-use connections between BT's heterogeneous systems.

4.1 SISS overview

The systems interconnect service shell [3] has been designed to keep the applications interface for both MVS/CICS and Unix as simple as possible, so that applications programmers need no specialist inter-system communications skills. Interfaces can then be written quickly and cheaply using standard and reliable code. To send a message via SISS from MVS, a CICS/COBOL application simply calls a SISS sub-module. Using a standard COBOL copybook, the application passes the text of the message together with information such as the target system and application, and whether or not the message requires acknowledgement. SISS handles everything else. On Unix, the function 'SISS_SEND_MESSAGE' performs a similar role for C programs.

To receive messages in MVS/CICS, SISS run a CICS program that executes any requested application, passing data via a standard COBOL copybook. On Unix, an already-running SISS transaction process can receive a message. Alternatively, there is an option to wake up a known service (via INetD).

The SISS architecture in Fig 3 provides for maximum flexibility when implementing an interface. A Unix system that needs to interface to (say) all 29 CSS systems does not necessarily need to be connected to all 29 systems but instead could connect to one MVS SISS host system which will route the data through to the correct machine.

4.2 Business benefits

The benefits to BT's business are listed below:

- the simple interface makes developers more productive by connecting systems quickly and easily,
- SISS can be reused across many applications, reducing development and support costs, and giving the benefits of upgrades and improvements to all users,
- SISS includes resilience and recovery features which help to reduce system downtime,

- the underlying communications protocol can be updated without impacting the application.

4.3 Technical features

The Unix code has been designed for maximum portability between different Unix platforms and has already been ported to Hewlett Packard and IBM RS6000 Unix machines from the development platform of Sun.

The list below summarises what SISS provides for the application or deals with on its behalf:

- simple API for CICS COBOL and Unix 'C' applications,
- protocol independent communications (currently SISS runs SNA LU 6.2) ,
- 'blocked' and 'unblocked' API options,
- validation of program requests,
- set-up and release of associations between sessions,
- system alias for remote systems and translation to system id and network address,
- network failure detection, and retry facilities to connect to a remote SISS if a failure is detected,
- automated recovery of communications links,
- configurable time-out facilities and wait time,
- synchronous MVS/Unix communications ,
- acknowledgement of requests,
- start up for different server processes depending on the 'operation' specified by the client application in the API,
- for COBOL developers on MVS it provides run unit control, so that the client application is informed of server errors — it will roll back the database if SISS times out and it can retry on certain errors such as database deadlocks,

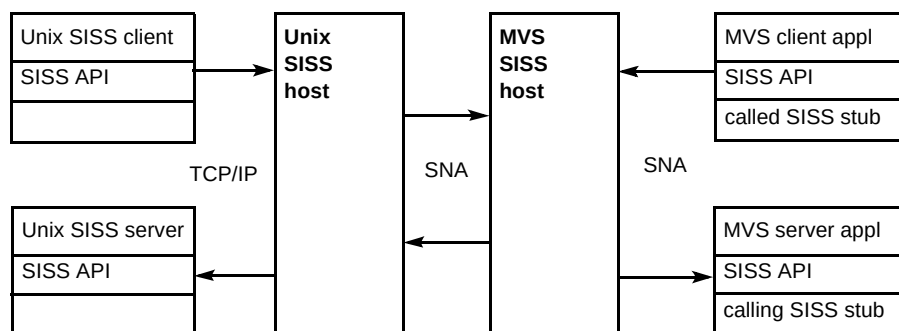


Fig 3 The SISS architecture.

- elementary routing to the correct application's CICS system, based on application-supplied data.

SISS also provides benefits in the operational area:

- handles multiple concurrent requests,
- performs logging for performance tuning, audit, diagnosis, exception and alert conditions,
- automates reconnection when communications are temporarily lost,
- has a low system overhead on Unix — SISS only runs when required,
- provides an operator interface for management and control.

4.4 SISS current status

Platform support — SISS is available on the following platforms: Sun Solaris 1.0 and 2.x, HP-UX 9 & 10, AIX 3.2.5 and MVS CICS 3.3

Production interfaces — SISS is now in production use for a number of interfaces. These projects are shown in Table 1 which details the platforms used for the host and API components, and the MVS application that is accessed.

Table 1 SISS interfaces.

Project (Unix application)	SISS Host O/S	SISS API O/S	MVS Application
RACE	Sun Solaris 1.0	Solaris 1.0	CSS
WMS	HPUX 9	HPUX 9	CSS
Telemetry	HPUX 9	HPUX 10	CSS
SMART	HPUX 9	HPUX 9	CSS
Raft	HPUX 9	HPUX 9	CSS
ACE	HPUX 10	HPUX 10	CSS
Billing 90s	RS6000 AIX	RS6000 AIX	CBP
D-Channel (CSP)	HPUX 9	Solaris 2.4	COSMOSS

4.5 SISS summary

SISS performs an important tactical role in BT providing a reusable MVS/Unix interworking solution. The separation between SISS API and SISS host functionality means that SISS access can be provided as an infrastructure component enjoying central support for operational running. In time SISS will be replaced with industry standard solutions (i.e. DCE RPC), but to date it has proved itself the only acceptable access for synchronous MVS/Unix communications in the BT environment.

5. Middleware message-based interface (MMBI)

Middleware systems account for a large percentage of BT's mainframe processing; they use over 2000 on-line programs and over 4000 on-line screens (maps). This represents a huge investment in development and a vast amount of customer-handling functionality.

In recent years the thrust within BT has been to reuse this existing functionality but to add extra functionality on non-Middleware open systems platforms. Too often the interface to the existing functionality on Middleware systems was executed at a screen level, i.e. the remote system emulated a terminal user's key strokes in order to execute a particular transaction.

This technique has become known as 'screen scraping' and although it provides a low-cost development interface to Middleware systems it also suffers from some well-documented drawbacks regarding its performance, maintainability and its dependence on a static screen format.

Middleware systems have suffered at the hands of screen scrapers with occasionally disastrous results. When a major application (SMART [4]) needed to provide a different interface to some 300 CSS transactions for 6000 users it was decided that screen scraping was not suitable and that an alternative development would be needed. MMBI was designed with the requirements of this application in mind [5].

5.1 MMBI overview

MMBI, like transaction switching described earlier in section 3, exploits the fact that Middleware applications do not handle input and output operations themselves. Instead this function is provided by Middleware itself. This separation has allowed the Middleware developers to accept requests for processing to be in a message-based format rather than a screen-based format and convert it into the data structure expected by the application. Likewise, after the application has executed, Middleware converts the returned data back into a message format before returning it to the remote application that made the request.

Figure 4 explains how MMBI can be accessed, in particular showing how access to legacy transactions can be executed over an existing 3270 connection, in which case Middleware is accessed directly or via an LU6.2 connection such as SISS, where the generalised message interface (see section 6) is accessed first.

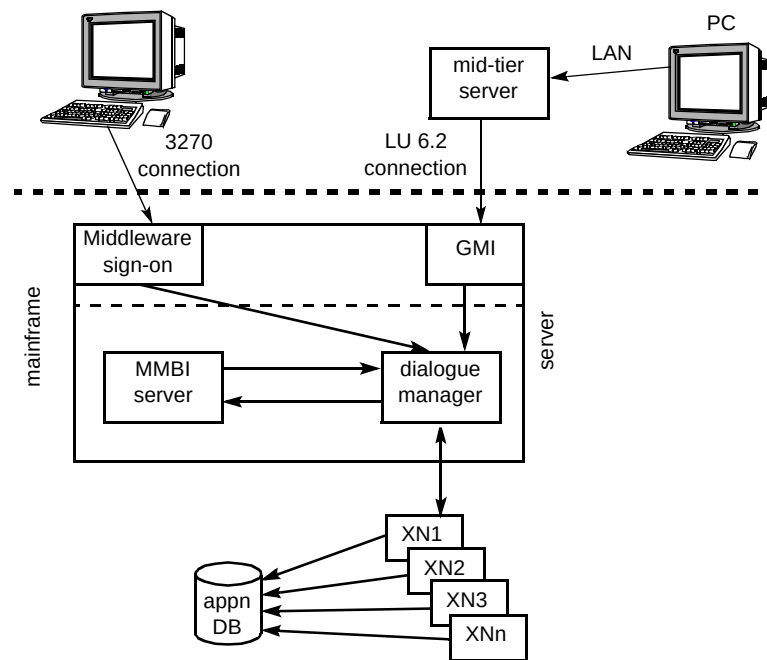


Fig 4 Access to the MMBI.

The MMBI enforces a particular type of self-defining data stream for the input and output message formats [6]. This message format is known as type-length-value (TLV) and allows a greater degree of flexibility in the interface between client and server

For example a TLV request message to run an MMBI object would look like:

```
MMBI RUN-OBJ;6;SAMPLE;CTL-END;1;*;TEL-
NR;10;0375814881 ;END;1*;
```

and the response from the Middleware system after the screen data had been converted into an MMBI message format might be:

```
MMBI OBJ-RSLT;6;SAMPLE;STS;2;OK;OBJ-
VER;1;1;DICTNY-VER;1;1;LCTN-ID;2;XX;CTL-
END;1;*;CUST-NM(1);16;JONES & HILT PLC;CUST-
ADDR(1);14;200 BAULK LANE;CUST-
ADDR(2);5;SULLY;CUST-ADDR(3);9;RINGSHIRE;CUST-
ADDR(4);8;RF19 3CH;TEL-NR;12;037 581 4881;RCNT-
ORD;3;YES;CURR-FAULT;2;NO;CPLNT;2;NO;FU-
RTNG;1;D;EXCH-NM;5;SULLY;OFF-SW-CHGNG-
IND;2;NO;HIST-FAULT;2;NO;END;1*;
```

5.2 Summary of MMBI objectives

There are four key target areas set for MMBI:

- to reduce the configuration management problems inherent in the use of screen-scraped mainframe applications by the ability to refer to data on a screen by name rather than position,

- to create the conditions to allow access to screen-based legacy applications by clients that are not display devices (e.g. other computer systems),
- to encapsulate the mainframe business applications via the use of a packaging mechanism (i.e. an MMBI object), both to simplify the running of complex business processes and to provide performance gains,
- to reuse code, allowing the use of legacy transactions in new and different ways, thus reducing the need to invest in new systems.

5.3 Components of MMBI

There are three key components of the MMBI:

- object scripts are collections of logic statements which together control the execution of existing Middleware transactions, providing the ability to manipulate the information being passed to or extracted from an individual business transaction,
- map views are generated as a result of processing the mainframe screen maps for each business transaction to be included in an object script, and allow MMBIs to form an association between a data element in the TLV message and the internal data representation, as known to the screen-based transactions,
- the standard name dictionary (SND) is held in an LDS (linear data set, a type of file) and defines all the message elements that are supported by the MMBI server together with various message data field

attributes; every database field that is displayed on a map accessed over MMBI has an entry in this dictionary.

5.4 MMBI — what are the advantages?

The benefits of using MMBI are sixfold, and the following list summarises these advantages.

- Information is passed in the form of a device-independent message. This replaces the traditional 3270 map and releases the front-end system from the need to know actual screen co-ordinates for data fields output by business transactions. These data fields become components in the message.
- Front-end system development aspects are simplified by allowing access to mainframe applications without the need for specific or detailed knowledge of how those applications are driven. This allows the definition of higher-level business functions through the packaging of multiple lower-level transactions and moves towards a business object approach. By hiding the majority of the complexities of running these back-end business processes, MMBI objects allow back-end transaction changes to be absorbed without disrupting the information delivered to the front-end.
- Performance is improved by running several mainframe transactions as an object rather than as separate individual transactions. Running an object requires only one client/server interaction regardless of the number of actual back-end transactions this may result in. Therefore the network component of response time is considerably reduced. Objects employ a language which allows selective control of information extracted from the transaction output, so only a subset of business information (i.e. only the data required by the front-end) need be included in the client-response message, thus potentially reducing the total amount of information flowing across the network.
- By enabling, through code reuse, access to standard transactions, which were originally written to inter-work with dumb terminals, the current investment in legacy applications is preserved and extended.
- As MMBI provides the technology to give access to existing applications, it can also provide future proofing by limiting the impact (on MMBI clients) of changes to those applications. Also, by presenting a consistent interface to the client, it allows back-end business processes to be re-engineered over time.
- In the event of a front-end failure or if it is not designed to totally 'front' the legacy system, MMBI/3270 provides, in the interests of safety, a mode-switch capability which allows PC front-ends to switch from 'message' mode to traditional 'screen' mode while

preserving the business-processing 'context' established by transactions which were previously executed in MMBI message mode. This mode switch causes all front-end/back-end interactions to revert to full-screen presentation used by CSS and other mainframe 3270 transactions. A mode switch back to message mode is also supported.

5.5 Summary

MMBI applications are able to interface to existing Middleware transactions over a robust and reliable network protocol. Middleware transactions can be run in sequence with extra logic being applied via an MMBI object script. Greater performance is achieved by batching these transactions to be run as a single process in the Middleware system. Greater reliability is achieved as well because screens susceptible to change are insulated behind the TLV message format and screen changes accommodated automatically by the map views.

6. General message interface (GMI)

Section 4 has already discussed the SISS product which provides a simple communications transport and API for MVS/Unix communications. SISS is an in-house product and BT strategy is now to use commercially available industry standard products wherever possible. The use of other products to get access to Middleware systems is therefore going to happen soon. The GMI is an attempt to bring together the various interconnectivity options in a way which provides the business applications with a technology-proof solution without limiting the horizons of existing business software [7].

The GMI objective is to offer a common interface for applications to gain access to messages passed over a variety of different delivery mechanisms (see Fig 5). Its aim is to hide the delivery of messages (by whatever mechanism) from business applications software, by providing a Middleware between the applications and the technology that connects them. It realises this in the following ways:

- an applications programming interface (API) to enable partner applications to exchange messages without knowledge of the underlying technology used to transport those messages,
- a network server component to provide a mapping layer between the applications and various transport products — it allows such transports as native LU6.2,

TCP/IP sockets, messaging services (e.g. MQSeries) to be incorporated without the application code being aware of the technology involved.

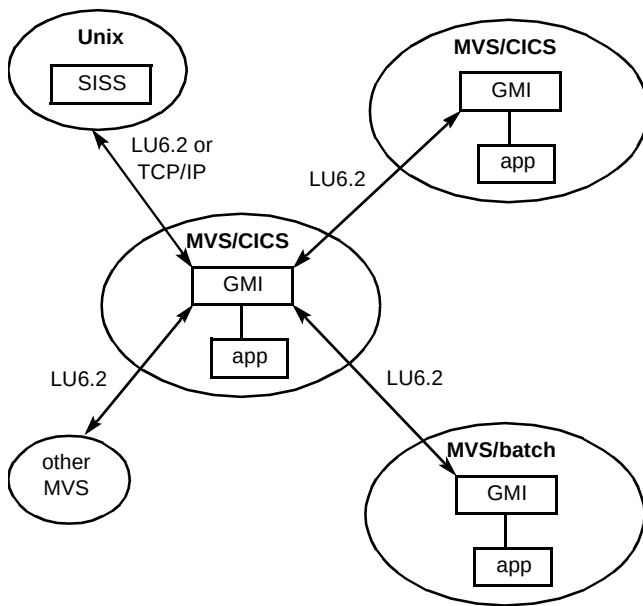


Fig 5 The GMI as a common interface.

The GMI is an enabling technology which provides a consistent way of passing messages between applications connected across a network. This enables the most appropriate 'transport' service to be used to provide the message-carrying capability. Currently, the transport services supported via GMI-api are:

- systems interconnect services shell (SISS), which connects Unix applications to MVS/CICS applications,
- Middleware message service (MMS) which connects MVS/CICS to MVS/CICS using LU6.2 protocols and CPI-C protocols.

The service can be used with GMI acting as the applications client and server at both 'ends' of a partner/partner interaction or just at one end (e.g. a client or a server) by mapping the GMI-api calls on to the selected transport.

The product does not replace the functions of message-transporting software, it merely 'fronts' them by hiding the specific technology. The choice of message-carrying technology can then be made on a 'most appropriate' basis. By separating the application's needs from the technical aspects of ever-changing interconnect technologies, the GMI provides a powerful Middleware solution — escaping from the costs of software vendor 'lock-in' and isolating the technology choices into an infrastructure component.

The GMI server component allows the message to be managed into the application environment in a consistent

(from the target application's viewpoint) way. The flexibility offered by this approach is illustrated in Fig 6. The set of transport adaptors (which understand the individual transport APIs) can be extended to encompass a variety of technologies. Figure 6 shows the range of access technologies that have been proven to work with GMI either in production or in prototype.

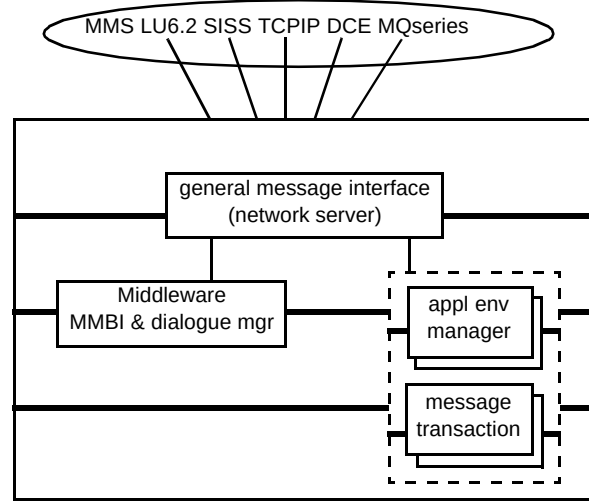


Fig 6 Access technologies working with GMI.

As other transports are needed, the GMI inter-product mapper will be extended to incorporate them.

The application processes are exposed only to the GMI and are not aware of the underlying transport service being used to carry the message between systems.

GMI operates in both a CICS/ESA and MVS batch environment. For batch, the transport is implemented using the open standard interface, CPI-C (via APPC and ASCH).

In summary, the GMI will continue to protect BT's investment in OSS systems by providing a seamless migration path between current tactical and long-term strategic interworking solutions.

7. Balanced distribution service (BDS)

The implementation of more and more applications on large numbers of small machines has led to an increase in the movement of data between BT's computer systems. Much of this data exchange is performed using file transfers and is of critical importance to the company. Much of this paper has been concerned with developing or providing access to on-line applications; BDS provides the same benefits as BT's Middleware for file-transfer interfaces [8].

There are several main file-transfer protocols in use within BT (OSI FTAM, TCP/IP FTP, NetView-FTP, Blue Book FTF, etc), but so far none of these has provided the

required level of functionality and cross-system availability to become the natural recommended protocol. It is unlikely that a single such protocol will emerge in the near future. Neither is there a single method of managing file transfers to ensure that data is moved when and where it is needed, and to present a consistent view of file transfer to its users, the applications.

This has meant that, until recently, application designers had often to include specific interfaces to file-transfer software in their programs; they also had to cater for matters such as error-analysis, recovery, routing and receipting — things that are more appropriate to communications software than business applications.

7.1 Overview

BDS is a process implemented to manage file transfers required for the Billing 90s 'off switch charging' and 'elapsed time charging' projects. It is currently available on Unix, Open VMS and MVS operating systems, using OSI FTAM and TCP/IP FTP file-transfer protocols. It provides a standard interface to applications wishing to transfer files between such environments. While originally developed for use by Billing 90s, it is not confined to any particular application.

The idea of a file-transfer manager to shield applications from underlying file transfer protocols is not new. It has obvious benefits in terms of easing support and minimising the changes to applications in the event of protocol changes. Billing 90s has been an enabler in getting this work started.

BDS offers two services to applications — data transfer and messaging.

7.2 Data transfer

An application requests BDS to get data from a remote system by identifying itself to BDS and requesting a poll service, stating what sort of data it requires, as shown in Fig 7. BDS uses a polling mechanism to ask remote systems for lists of files which contain that data; the list is returned as a poll response, and includes a checksum value and size for each file. BDS automatically transfers each file in the list, translating if required, and performing another checksum on receipt. It is this second checksum which ensures data integrity. Any data file which fails to transfer is 'pulled' again. After all file transfers are complete, an acknowledgement is returned to the source system; it contains a copy of the original list of files from the poll response, with a marker against each file to show whether it was transferred successfully or not (and, if not, why not). A copy of this

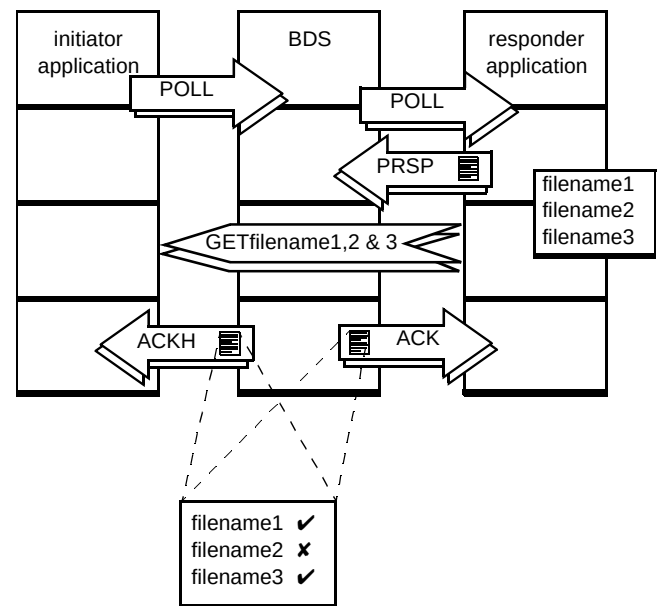


Fig 7 The BDS data transfer application.

receipt is also returned to the application which requested the data in the first place, the filenames being converted to the local system's format.

Refinements to this protocol allow BDS to suspend the mechanism selectively for short periods, e.g. when the data required is in use by another application.

To cater for applications with data destined for other systems, BDS can be asked to send an alarm to a remote system. The alarm indicates that the remote application should poll the system sending the alarm for the data.

7.3 Messaging service

BDS provides a simple messaging service for applications. It can be asked to send a message in a PCI file to a remote system. A response from the remote system confirms receipt.

7.4 BDS implementation

There are two main implementations of BDS — a Unix implementation written in 'C' by BT's Belfast Engineering Centre, and an MVS implementation written primarily in REXX. In the Unix-based BDS, applications can interface to BDS by calling routines supplied in a BDS interface library; communication between the processes are via sockets. On MVS, applications supply parameters to a supplied BDS interface batch job; BDS for MVS can be configured to submit specific application batch jobs on receipt of requests from remote systems.

7.5 BDS summary

The move towards distributed systems has increased the demand for file transfer as well as on-line access. The balanced distribution service provides a consistent and manageable file-transfer infrastructure for all BT's application platforms and environments. BDS has been extremely successful and is in use on more than 50 system interfaces. Its main strength lies in its ability to use the most appropriate protocol (FTAM, FTP, etc) transparently to the application while providing value-added features via a programmatic interface.

In the future BDS will be enhanced to use other industry/*de facto* standard solutions, the most prominent of these being, 'CONNECT: Direct' from Sterling Systems.

8. Software as infrastructure

The Middleware components described in this paper have evolved over a number of years in response to real business requirements. The fact they complement each other and comprise an integrated portfolio of solutions is partly due to the fact that they have all been developed within BT's Systems Engineering Centres, but also because of the initial structure of BT Middleware.

The isolation/insulation of application programs from the environment and the reuse of common functions across multiple applications allows the introduction of new solutions and technologies transparently to the applications themselves. This structure that Middleware imposes enables software to be viewed as infrastructure in the same way that a network or an operating system can be viewed as infrastructure.

It has been said of BT's Middleware components that they are simply 're-invented' basic functions of the underlying operating system software. However, the Middleware components described here actually represent a thin layer (see Fig 8) of software over that basic functionality that makes use of the underlying functions, wherever possible only adding features that are not supplied.

For example, SISS does not provide its own network protocol but reuses an industry standard (LU6.2) protocol while greatly simplifying the interface for the programmer and adding dynamic configuration and routing capabilities.

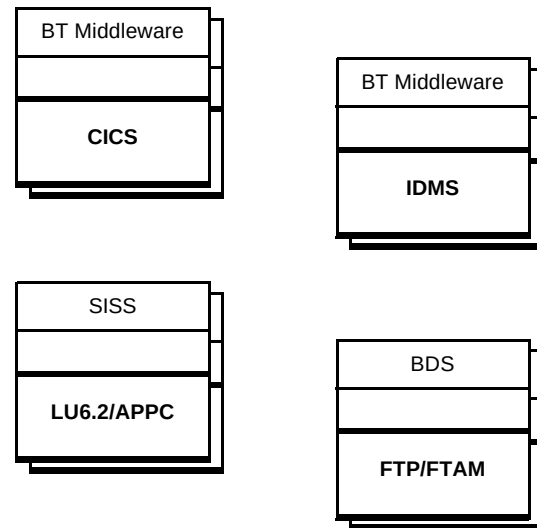


Fig 8 Middleware software components as infrastructure.

9. BT Middleware — the future

The future of BT Middleware, like that of BT's OSS systems, in general lies in extending the ability to distribute applications easily and quickly, giving access to the required business functions in the required manner. This means Middleware must be able to integrate with all new strategic technologies such as OSF/DCE and MQSeries, through the GMI rather than the existing in-house developed solutions. Exploiting the services of commercially available products will mean a thinner layer of Middleware is used.

Middleware systems also need to be able to participate in an object-oriented world more closely than they do today. This means integration with distributed object technologies and the use of object-oriented programming techniques in the mainframe environment.

Direct customer access to BT's OSS systems via the World Wide Web is also a possibility that may require the capabilities of Middleware to be enhanced. Because of the applications environment that has been provided, Middleware systems are uniquely positioned to meet the challenge set by new technologies and business opportunities into the next century.

Acknowledgements

The software described here has been produced over a number of years of hard work by the people in the Systems Interconnect and Middleware team from the London Software Engineering Centre. The author gratefully acknowledges the contributions of Steve Barnes, Barry Darville (in particular for providing the content of section 7), Peter Kennedy and Colin Richards in the production of this paper.

References

- 1 Harrison P F: 'Customer service system — past, present and future', BT Technol J, 15, No 1, pp 29—45 (January 1997).
 - 2 Chandler J: 'Management of special services — designing for a changing world', BT Technol J, 15, No 1, pp 86—96 (January 1997).
 - 3 Richards C: 'SISS Overview', Internal BT document (1994).
 - 4 Selley C J et al: 'SMART — improving customer service', BT Technol J, 15, No 1, pp 69—79 (January 1997).
 - 5 Barnes S J: 'MMBI Overview', Internal BT document (1994).
 - 6 Calladine J and Kennedy P: 'MMBI Message Format Specification', Internal BT document (1994).
 - 7 Kennedy P: 'GMI Overview', Internal BT document (1995).
 - 8 Darville B: 'BDS Overview', Internal BT document (1996).
-



Jonathan Calladine graduated from Sussex University in 1986 with a degree in Philosophy and Artificial Intelligence. He joined BT in 1987 working as a CICS systems programmer on the CSS project.

In 1992 he moved to the Systems Interconnect and Middleware team working as a consultant and defining BT's technical strategy for distributed applications.